Figure D1: Core PyTorch Implementation Snippets of K-FedICGL

Note: *Strictly complying with the UAI policy restricting external links to only figures and tables, we provide the core implementation logic as a supplementary algorithmic figure. These snippets illustrate the exact mathematical and tensor logic of our framework. The complete, end-to-end training pipeline will be fully open-sourced upon acceptance.*

Snippet 1: Local Discovery (LocalCausalGraphGenerator.py)

```python
import torch
import torch.nn as nn

class LocalCausalGraphGenerator(nn.Module):
    """
    Client-side Knowledge-Driven Causal Graph Construction (Section 3.2).
    Learns a local adjacency matrix constrained by physical road network priors and DAG properties.
    """
    def __init__(self, num_nodes, lambda_prior=0.1):
        super().__init__()
        self.num_nodes = num_nodes
        self.lambda_prior = lambda_prior

        # Learnable continuous adjacency matrix A^(k) initialized near zero
        self.A = nn.Parameter(torch.zeros(num_nodes, num_nodes))

    def _compute_dag_constraint(self):
        """
        Computes the trace constraint h(A) = tr(e^(A o A)) - N = 0 to ensure a DAG structure.
        """
        A_sq = self.A * self.A  # Hadamard product A o A ensures non-negativity
        E = torch.matrix_exp(A_sq)
        h = torch.trace(E) - self.num_nodes
        return h

    def compute_local_discovery_loss(self, L_rec, K_prior, rho, alpha):
        """
        Computes the Augmented Lagrangian loss for causal discovery (Eq. 4).
        """
        # Step 1: Prior Penalty Mask M = 1 - K (Eq. 3)
        M = 1.0 - K_prior

        # Frobenius norm of Hadamard product imposes penalty on spurious edges violating priors
        L_prior = torch.norm(self.A * M, p='fro') ** 2

        # Step 2: DAG Constraint h(A)
        h = self._compute_dag_constraint()

        # Step 3: Total Augmented Lagrangian Loss (Eq. 4)
        total_loss = L_rec + (self.lambda_prior * L_prior) + (0.5 * rho * h ** 2) + (alpha * h)

        return total_loss, h
```

Snippet 2: Server Aggregation (server_aggregation.py)

```python
import torch
import torch.nn as nn

class FederatedInvariantAlignment(nn.Module):
```

```python
    """
    Server-side Invariant Causal Subgraph Alignment (Section 3.3).
    Maps independent client subgraphs into a Common Node Space for consensus and variance.
    """
    def __init__(self, num_global_nodes, tau_c=0.5, gamma=1.0, tau_s=0.5):
        super().__init__()
        self.N = num_global_nodes
        self.tau_c = tau_c  # Threshold for consensus (Eq. 5)
        self.gamma = gamma  # Penalty coefficient for variance (Eq. 6)
        self.tau_s = tau_s  # Threshold for final invariant mask (Eq. 7)

    def forward(self, client_adj_list, client_node_indices):
        num_clients = len(client_adj_list)

        # Step 1: Map local subgraphs to the Global Common Node Space
        global_A_stacked = torch.zeros((num_clients, self.N, self.N))
        observation_mask = torch.zeros((num_clients, self.N, self.N))

        for k in range(num_clients):
            local_A = client_adj_list[k]
            idx = client_node_indices[k]
            idx_grid_x, idx_grid_y = torch.meshgrid(torch.tensor(idx), torch.tensor(idx), indexing='ij')
            global_A_stacked[k, idx_grid_x, idx_grid_y] = local_A
            observation_mask[k, idx_grid_x, idx_grid_y] = 1.0

        # Step 2: Compute valid average across clients (ignoring unobserved edges)
        num_observations = observation_mask.sum(dim=0).clamp(min=1e-5)
        A_mean = (global_A_stacked * observation_mask).sum(dim=0) / num_observations

        # Step 3: Compute Structural Consensus Matrix C (Eq. 5)
        is_above_threshold = (global_A_stacked > self.tau_c).float() * observation_mask
        C = is_above_threshold.sum(dim=0) / num_observations

        # Step 4: Compute Cross-environment Variance V (Eq. 6)
        squared_diff = ((global_A_stacked - A_mean.unsqueeze(0)) ** 2) * observation_mask
        V = squared_diff.sum(dim=0) / num_observations

        # Step 5 & 6: Active Alignment Score S and Invariant Skeleton A_inv
        S = C * torch.exp(-self.gamma * V)
        M_inv = (S > self.tau_s).float()
        A_inv = M_inv * A_mean
        A_inv = A_inv * (num_observations > 1e-5).float() # Zero out completely unobserved

        return A_inv
```

Snippet 3: Local Optimization (CounterfactualContrastiveOptimizer.py)

```python
import torch.nn.functional as F
import torch.nn as nn
import torch

class CounterfactualContrastiveOptimizer(nn.Module):
    """
    Client-side Counterfactual Contrastive Learning (Section 3.4).
    Implements intervention-inspired augmentation (Eq. 8).
    """
    def __init__(self, tau_nce=0.5):
        super().__init__()
        self.tau_nce = tau_nce

    def forward_intervention(self, gnn_encoder, X, A_local, A_inv, noise_std=0.1):
        # Step 1: Decouple spurious local edges A_spur = max(0, A_local - A_inv)
        A_spur = F.relu(A_local - A_inv)

        # Step 2: Generate Gaussian random noise epsilon ~ N(0, sigma^2 I)
        epsilon = torch.randn_like(X) * noise_std

        # Step 3: Counterfactual Representation Generation (Eq. 8)
        Z_inv = gnn_encoder(X, A_inv)            # Anchor: Pure causal paths
        Z_noise = gnn_encoder(epsilon, A_spur)   # Spurious path driven by noise
        Z_cf = Z_inv + Z_noise                   # Counterfactual positive sample

        return Z_inv, Z_cf

    def compute_infonce_loss(self, Z_inv, Z_cf):
        """ Computes the Causal Graph Contrastive Loss (Eq. 9). """
        Z_inv_norm = F.normalize(Z_inv, p=2, dim=-1)
        Z_cf_norm = F.normalize(Z_cf, p=2, dim=-1)

        # Compute cosine similarity matrix
        similarity_matrix = torch.matmul(Z_inv_norm, Z_cf_norm.transpose(-2, -1)) / self.tau_nce

        N_k = Z_inv.shape[0]
        labels = torch.arange(N_k, device=Z_inv.device)

        # InfoNCE loss
        L_cgc = F.cross_entropy(similarity_matrix, labels)
        return L_cgc
```

Snippet 4: Causal Evaluation Metrics (evaluation_metrics.py)

```python
import numpy as np

def build_pseudo_causal_ground_truth(sensor_distances, distance_threshold=2000.0, k_nearest=5):
    """
    Constructs a pseudo-causal ground truth graph for traffic datasets based on physical
    spatial proximity (serving as the reference graph for SHD, FDR, and TPR evaluation).
    """
    N = sensor_distances.shape[0]
    A_true = np.zeros((N, N))

    for i in range(N):
        valid_neighbors = np.where(sensor_distances[i] <= distance_threshold)[0]
        if len(valid_neighbors) > k_nearest + 1:
            sorted_idx = np.argsort(sensor_distances[i, valid_neighbors])
            valid_neighbors = valid_neighbors[sorted_idx[:k_nearest + 1]]

        for j in valid_neighbors:
            if i != j:
                A_true[i, j] = 1.0 # Directed physical flow assumption
    return A_true

```

```python
def evaluate_causal_discovery(A_pred, A_true, threshold=0.1):
    A_pred_bin = (A_pred > threshold).astype(int)
    A_true_bin = A_true.astype(int)

    shd = np.sum(np.abs(A_pred_bin - A_true_bin)) # Structural Hamming Distance

    true_positives = np.sum((A_pred_bin == 1) & (A_true_bin == 1))
    actual_positives = np.sum(A_true_bin == 1)
    tpr = true_positives / actual_positives if actual_positives > 0 else 0.0

    false_positives = np.sum((A_pred_bin == 1) & (A_true_bin == 0))
    predicted_positives = np.sum(A_pred_bin == 1)
    fdr = false_positives / predicted_positives if predicted_positives > 0 else 0.0

    return {"SHD": shd, "TPR": tpr, "FDR": fdr}
```